Pega.dotx.docx

APPLICATION CREATE FROM SCRATCH: 7.4

#1 Analyse and come up with ECS (Enterprise Class Structure)

#2 Create Framework application —> go to New Application and create

#3 Create Implementation application —> go to New Application and create

#4 Identify and Create Case Types —> Add Case Type

#5 Identify and Create Data Types —> Add Data Type

#6 Create Data Model and UI Wireframes and link them on to flows

#7 Code Logic: GET, SAVE

#8 Create Operators

#9 Configure Security : {Map Access Role, Access Role To Object}:

#10 Create actual case from the user portal

APPLICATION CREATE FROM SCRATCH: 8.4:

From Application menu click on New Application

Select Custom —> Use this application type —> Enter name of your application —> Click Advanced configuration

Application setting select FRAMEWORK / IMPLEMENTATION —> Application id, version —> Select Organisation settings like Org, Division and unit name —> Class layers like Org-Division-Application-Work and then click save.

Click on create application and click Goto App button.

######### ########## #########

$$$ What is new in Version 8:

1. There are 4 new features added Pega 8, Dev Studio (Designer studio in 7), App studio, Admin studio (SMA moved under this), Predictive studio (Data scientist and AI team).
2. Replaces Standard Agents with Queue Processors and Advanced Agents with Job schedulers

3. Cosmos application provides presentation layer with information architecture, interactions and proper accessiblity.

$$$ What is Application Architecture:

1. When ever user hit the URL in the browser login screen will display
2. It will verify the User id and Password within the LDAP authentication server.
3. Once it verifies the credentials then it will take you to the web server based on the URL given
4. From Web server it will take you to the Application server and it will read WEB.XML file. here, It will read the PRSERVLET.
5. PRSERVLET will have authentication service. Auth service will have authentication activity (pzOAuthAuthorisationActivity). In this activity it will do OBJ-OPEN on the Operator table.
6. Post that Operator will have access group, application, roles and ARO etc

$$$ Work list & Work basket.

1. Work list means specific to operator and we can see complete list of work objects.
2. Work basket means specific to group of operators that we will define in work group.

$$$ How to remove duplicates from Pagelist $$$

1. Obj-browse to fetch the records and put them on Pagelist.
2. Remove the duplicates from the Pagelist using below JAVA code.

```
If(myStepPage != null)
{
        ClipboardProperty pyStateListCase =
myStepPage.getProperty("pxResults");
        Pega_rules_Utilities.removeDuplicateFromList(myStepPage,
"UserName", "Rule-Obj-FieldValue");
}
```

$$$ How to UPDATE GOAL AND DEADLINE FOR SLA $$$

Please use pxAdjustSLATimes final API activity for use case. Pass all the adjusted values to this activity as parameter.

1. SLA is used to complete the task within the given time.
2. Goal and deadline time runs only once.
3. You can reuse SLA's in multiple places like assignments, processes, stages and even on complete case type.
4. If SLA doesn't met Goal, Deadline and Passed deadline then in that case we can have a separate escalation process.
5. To manage these escalation we have OOTB process Agent Pega-ProCom.

$$$ How to configure SLA's $$$
SLA events are written to Queue class and processed by an OOTB agent ServiceLevelEvents of ruleset Pega-ProCom.

$$$ How are events written to Queue class $$$
During SLA invocation, system takes care of writing an entry for Goal event into queue class which is mapped to table pr_sys_queue_sla.
Standard agent activity "ProcessEvent" takes care of creating a new entry for deadline during Goal event processing.

$$$ Child Cases: $$$
Data Propagation means to copy the data from pyWorkPage to Primary page.
"pxAddChildWork" is the activity to create child case.
"pxCoverInsKey" is the property to hold Parent case key
"pxCoveredInsKey" is the ValueList to hold Child case key

There is no OOTB way to Automatically propagate Data from a Child case to its Parent Case. You will need to write a Custom Data Transform or Activity to propagate changes from a Child Case to its Parent Case and then call the Custom Data Transform or Activity when you need to propagate the data back to the Parent.

SCEN 1: I have parent case P1 which has default locking configured and has child case C1. If I lock P1 will it lock C1?
Sol: In this case child case objects will also be locked.
SCEN 2: If I lock C1 will it lock P1?
Sol: It depends on the setting you have chosen child case whether to lock the parent case or not.
Case type —> Advanced tab —> "Do not lock the Parent case when the Child case is being performed" check box

$$$ Why We need Queue Processor when we already have job scheduler? $$$
Job scheduler runs on specified time or some specific time interval
Queue processor runs when only entry had been made and also it has an advantage of broken queue management.

$$$ Standard and Advanced Agents $$$
Standard:
1. For ProessFCPCase we can use Standard agent.
1. It queue the items. Whenever write anything in the Agent it will first taken by the Master agent and it will be queued into the items and then it start executing.
1. It uses access group of user who queues the entry.
2. It supports AQM
3. It wake and run only when there are queue entries with available locks.

Advanced:

1. For CreateFCPCase we can use Advanced agent. Queue-For-Agent method is used to the queue the items and need to pass parameter as standard agent.
    1. There will be no queuing things. It will directly executed.
    1. It uses the access group specified in the agent rule
    2. It do not support AQM
    3. It runs when there are no queue entries this is because advanced agent responsible for AQM

$$$ How to create work object in AGENT:

1. Each ruleset will have single Agent. Agent is associate with an activity.
2. For creating work object we can choose Advanced Agent and for processing we need to choose Standard for locking mechanism.
3. Once we create and save agent our Agent Schedule gets created.
4. Create Page-new in agent class —> do property-set —> call svcAddWorkObject activity and pass params flow type, Org, work page.

$$$ GLOBAL RESOURCE SETTINGS: $$$

HOW TO CONFIGURE URL'S GLOBALLY IN OUR APPLICATION WITHOUT CHANGING MANUALLY:

Expectation: If user login with DEV url only DEV url should be loaded like wise if QA url QA url should be loaded.

1. Create DataPage with class DATA-SERVICECONFIG in NODE level scope with DATA SOURCE as activity.

Configuration of Data Type should be like,

SERVICE NAME	PRODUCTION TYPE	SERVICE URL
service one	1	https://dev.com
service one	2	https://qa.com

2. Create VALUE GROUP property "EndPointURL" (SERVICE NAME) in DATA-SERVICECONFIG class. By using Value Group to bring the desired structure with key value pair.

EX: D_ServiceConfig.EndPoint(SERVICE NAME)

3. Write the business logic to fetch the results from DB.

OBJ-BROWSE to fetch the results from DATA-SERVICECONFIG class.

Do the PROPERTY-SET to set the ServiceName and URL thru loop the page.

4. We can use =D_ServiceConfig.EndPointURL(ServiceOne) in Connector rules.

$$$ DATA PAGE:$$$

$$Why do we refer Access group in Node level data page?

RRA works based on requestor access group. For node level we may have different requestors with different access groups per Node.

If access group specified in the load management tab then Pega uses the access to identify the data source activity using rule resolution and load data.

$$ If the data page is needed by 2 or more access groups, then we can have one Batch access group created with required application as built on and configure it to the node level data page.
$$ To flush the data page thru activity: @(Pega-RULES:DeclarePages).pzDeleteAllInstancesOfDeclarativePage(tools, Param.PageName).
$$ call "FlushDeclarativePage" Activity and pass page name as the parameter to be flushed along with parameter pyFlushAll as true.
$$ EDITABLE DATA PAGE:

1. Editable data pages doesn't have refresh strategy.
2. It load only once upon first reference and continue to use the same data for subsequent references. This allow us to modify the page without worrying about overwriting the data.
3. While data page can be editable they do not automatically persist back to the source. this must be done explicitly.
4. Editable data pages are displayed under USER PAGES where as Readonly data pages found under Data page in clipboard.
5. Editable mode is available only for thread and requestor because Thread and Requestor scope data page only applicable for particular requestor. They can update data and it would be applicable to that user. But Node level data pages are available for all the requestors in a particular node. So if we make it editable, it can be reflected to all users and comes with huge performance issue.

$$ Difference between Editable and Savable Data pages?
Editable – This page contains initial contents that can be accessed in read-write mode. Editable data pages do not have a refresh strategy or save plan and cannot be node-level in scope. These data pages are displayed in the user page list on the clipboard.
Example: To load Customer Profile information like FirstName, LastName, EmailID, NewCustomer, etc..
Savable – This page provides a save plan through a database source or an activity so that users can update data and write to a system of record. Only savable data pages can be referenced in save data page locations, which are areas in the application where you can list data pages to save. For example, you can list the pages to save in flow actions during postprocessing, the Save Data Page shape to use in Case Designer or the Flow form, and the Save-DataPage method to use in activities.
Example: To save the KYC details of a customer.

NODE: If we specify Page in Node level, then it can be accessed by all the requestors in the particular Node.
it is available for all the users.
EX: Bank has introduced a new scheme for all customers. In this scenario the scheme should be visible to all bank customer. So that we can specify Data Page as Node level scope.

REQUESTOR: If we specify Page in Requestor level, then it can be accessed by all Threads opened by the requestors.
It is specific to the particular user
EX: Personal details will be available for the specific person. Let's say if a customer login with CRN ID then he will be able to see only his personal details and bank transactions.

THREAD: If we specify Page in Thread level, then it can be accessed only by that particular thread.
One user can work on multiple thread
EX: We are going to mask the customer details for few operators in grid level. In that scenario we need to create DataPage in thread level and call an activity as data source. Based on the access roles when condition we will be masking data. If unauthorised operator login and check he can see the masked data in the grid.

$$$ KEYED PAGE ACCESS: $$$

This only applicable for Pagelist structure and Read only mode.
We have DataPage D_PolicyList (BOOK) contain different Policies (CHAPTERS). If we need to access the particular policy then we need to know the PolicyID (PAGE NUMBER) to access.
EX: Need to display the bank details based on the selection of the bank.

1. First we need to select Access pages with user defined keys check box in data page and select the pagelist name.
2. Create single page with Page def as data page class and select the radio button Refer to data page from data access.
3. Provide the data page name and pass the key value like pagename.property
4. On the UI provide the bank details like Pagename.bankname, Pagename.banklocation, Pagename.passcode
5. When we select the bank name from the drop down the bank details will be populated.

$$$ How to parameterised Data Page in activity/DT $$$
If the DP has two parameters then you need to specify like,
D_Customer[CustomerID:"123", CustomerName:"ABC"]
If the DP has only one parameter then you need to specify like,
D_Customer[.CustomerID]

$$ Difference between Keyed page access and Parameterised data page:

1. For Keyed page access once we refer the data page then data will be stored in clipboard.. whereas if we configure parameterised data page based on the parameter reference it needs to hit the DB.
2. Keyed page access does't maintain multiple instances of data page in

clipboard but the parameterised data page does.

$$$ Purpose of Post load processing in Data Page $$$
This activity is used to process the data page after loading. this is mainly used in error processing. If the data source returns invalid data, then we have logic here to process the errors.

1. Rule availability

Yes: It is available to RRA
No: For EX: 01-01-02 has correct code but my code has some issue that is available in 01-01-03. So we will make No to 01-01-03 then it will pick 01-01-02
Withdrawn: Let's say we have created same rule in ABC ruleset and XYZ ruleset. ABC ruleset related rule is not the correct rule to process. Then we will withdrawn ABC. So that the rule will be processed from XYZ ruleset.
For EX: the same rule is available in both ABC RS and XYZ RS. But the correct logic is available in XYZ but not in ABC.In that scenario we will withdrawn ABC.
Blocked: If RRA selects rule that is blocked, then that rule and all others(same name or key, any ruleset) are not executable.The rule will be called by RRA but throws an exception.
Final: We can save as the rule in the same ruleset not in other ruleset.

2. Refresh the data page means we need to call the data page then only it will refresh. If we specify the refresh strategy in data page it will refresh only when the data page invoke.

3. Sub Reports: Run a report definition means to generate single SQL query. If we want to create multiple queries like main query and sub query then we can go for Sub Reports. To satisfy the complex requirements and use the results of the existing report def we can use sub reports.
   Sub report is a name we refer to the report definition rule, when it is used in the another report def rule.
   EX: select pyid, pystatuswork form work class where pystatuswork in/ ==/ != { select pyassignment from pcassignworklist}

Req: To get the list of operators who don't have any assignment entry.

1. Create a new report def rule in Data-Admin-Operator-ID class.
2. run the report def and check the operators list.
3. Create another report def rule in Assign-Wortlist class (sub report) with pxAssignedOperatorID
4. run the report def to get the list of operators who have assignments in their pc-assign-worklist table.
5. Go to the main Operator report and add the sub report.
6. click on the configuration option and add the filter condition.
7. Now add the filter criteria in Operator report like, pyUserIdentifier != Assignment.Operator_ID

8. save and run the report.

JOINS:
Inner Join: It fetches the rows which are common in two tables.
Left Outer: it includes all rows in the left table and only uses data from the right table
Right Outer: Include all rows in the right table and use only data in the left table.
SCENARIO: To display Sales and Assignment details.
Sol: Prefix —> A, Class name —> Assign-Worklist, Type —> Inner join
Edit condition: Condition —> A0, Column —> pxInsKey, Relation —> Equals to, Value —> A.pxRefObjectKey

REPROTS:
$$ We can use unoptimised properties in Filter criteria but this will affect your performance.

How to fetch limited records from Report def:
$$ In the Activity Pages & Classes, pass pyReportParamPageName (Page name) & Class (Embed-QueryInputs)
$$ In Property-Set, set Report name, Report class, Report page and Param. pyReportParamPageName and value "pyReportParamPageName"
and Set —> pyReportParamPageName.MaxRecords —> Value as DSS.

IMP SCENARIOS IN REPORTS:
SCEN 1: I need to fetch and display recently created cases:
Sol: My ranking criteria will be used to fulfil the scenario.
SCEN2: Need to fetch and display the Operators who are getting the highest salary in their department.
Sol: You can enable 'For each group' and do some configuration in Report Viewer tab.
SCEN3: How to send report in email without using Agents:
Sol: We have Report Scheduling option in Case Manager portal.
$$ Pre-requisite for Report Scheduling:

1. A working email account
2. Agent ScheduledTaskProcessor (Pega-RuleEngine)is enabled.

$$ How to enable and scheduling a report:

1. Launch Case manager portal
2. In left navigation select REPORTS —> NEW REPORT
    1. Enable scheduling on the sales report

4. we can call activity in decision table by using Property-Map-Decision table. If we un check "allow missing properties" check box in activity passing parameters then if any of the input property missed in the decision table if else condition then it will throw error.

$$ In decision table IF/ELSE condition, if one IF condition satisfies then cursor will directly go to return step.

$$ If evaluate all rows check box is selected then it will return last when

condition.

5. Obj-save-cancel: Suppose we configure 5 obj-save methods are there in an activity. if we use obj-save-cancel it will remove last obj-save method from deferred list. if we use Rollback it will delete all obj-save methods from the deferred list.

6. connector-problem flow will call when we didn't define our own exception handling. If we define our own exception handling then it will call customised one.

$$ You can see when resource is unavailable it goes to IncompleteConnecton work basket and retries connection using SLA.

$$ If the exception is other than IncompleteConnecton then the spin off sub flow process will invoke.

7. Pattern Inheritance: a-b-c —> pattern form a-b like naming convention.
   $$ allows to reuse rules within single application.
   Direct Inheritance: means implementation class directly inherit from framework class.
   $$ allows to reuse rules in other applications, including standard rules provided as part of Pega platform.

8. Diff between Sub Report and Drill down Report:
   Sub reports can be run on the same or different reporting server but, drill down report must be run on the same reporting server.

9. Declare Expression: It Computes property value based on expression. This rule gets triggered based on two methods like forward and backward chaining.
   EX: We have DOB and AGE field. When ever user enter DOB field, we need to automatically compute Age value.
   —> Set Age == value of (birth year - current year)
   EX: Based on Cost and Quantity we need to compute the TotalPrice value.

Forward Chaining: When ever input changes target property gets calculated.
EX: TotalPrice == Cost * Quantity
Whenever Cost or Quantity changes DE fire and TotalCost gets calculated baed on the expression in the rule.

Backward Chaining: This exactly opposite to Forward chaining.
EX: TotalPrice == Cost * Quantity
Whenever the TotalCost is null, the DE fire and check for the input properties (Cost and Quantity)
imagine Cost is calculated as,
Cost == BasePrice - Discount

Now the DE tries to capture the Cost value if it is null.
It is like going backward and backward, until you get the required input values to calculate the target property.
NOTE: We can use Property-Seek-Value method to execute backward chaining. Parameters are Goal Property, Always recompute, AlwaysForwardChain, Missing Reference Property.

10. Declare Onchange: When the value of specific property changes then it can execute an activity.

EX: We need to track “Subtotal” property whenever the total purchase is above 10K then we need to add gift watch.

1. Add Subtotal property in DO rule.
2. Create When rule where Subtotal > 10K
3. Create new Onchange activity type and use Property-Set

11. Declare Trigger: When an instance of specific class is created, updated, deleted it will execute an activity.

EX: Let’s say we are going to buy a property. Buying a property involves many people to approve and adjust the TotalCost. Broker can set the amount as 10L but this amount will be adjusted by the owner and any other related parties. We have requirement to track the TotalCost in each stage who have update the cost.
Track the “TotalCost” property values in each page in the page list lineitem()

1. Create DT rule and trigger when an instance is save and one of the other properties modified.
2. TotalCost property value copy value to OldTotalCost
3. Specify the Page Context as Lineitem()
4. Whenever the TotalCost property in the grid gets updated and saved, the value is copied to the new property OldTotalCost

12. Defer Load Contents: some delay in loading some data in the initialisation of page we can use Defer Load. At the time of initialisation of screen delay in the data loading.

We can use repeating dynamic layout to load the content on screen we can use defer load. If we want to delay showing on the screen at a time we can use pagination select “Progressively load on user action”. based on the page selection we can load more data on the screen.

Another pagination is “Progressive load on scroll”. based on the scrolling we can see the data loading.

We can configure Defer load in Dynamic layout, Repeating grid layout, Table and Dropdown .

13. Spin off: It uses the parallel processing.

main flow can’t be completed until the sub flow completes.
that means, the current flow continues to executed, the spin off flow executed simultaniously.

For EX: if we take credit card application, agent will get the details from the customer and process it. bank officer will get create the object but the sub flow (Agent) process not completed yet. Sot we can't resolve completed the flow until the sub flow completes.
$$ Synchronous: It can execute only one process at a time.
$$ Asynchronous: It can execute multiple processes at a time which does not have any dependency.

14. Diff between Split-Join and Split-For-Each flows:

The approval process is different for Split-Join like Operational and Sales teams.
The approval process is same for Split-For-Each like common approval required for Sales, Seller and Operational.
If we select Join condition like any/all it would success for at least one join condition.
If we select iterate option we need to iterate thru Page list or Page group. Let's suppose if we select pyWorkParty (Page group) property it contains two Page like pyWorkParty(Sales) and pyWorkParty(Service). Based on the subscript the flow will execute.
EX: Scenario for Split-for-each is one candidate has to give interview with multiple interviewers.

1. First we need to create data class with data class properties.
2. we need to create work class with page definition as data class.
3. Create section and provide the interview related fields.
4. Create a flow in work class and define Split-for-each shape and open the properties panel and update the Page property, data class and flow name.
    1. On the UI if we add interview related fields it will iterate thru page list and create the assignments.

15. How to configure Connect-SOAP:

XML Request: Policy ID
XML Response: CustomerName (Single value), PolicyDetails (Page list), Address Details (Page)
Run the wizard then it will be landed in the wizard screen.

1. Load WSDL (Upload WSDL via URL or File) POST method
2. Select Operations. (WSDL service name, PORT name, EndPoint URL, Operations will be populated from the WSDL file)
3. Review. We can specify Integration class and RSV here

Once submit 5 types of rules will be created.
Class
Property
Connect-SOAP
Parse SML
XML Stream
$$ How to delete all rules created by the wizard?

Designer Studio —> Integration —> Tools —> Wizard clean up
We can call the Connect-SOAP rule in the activity to fetch CustomerDetails.
$$ Error Handling:

1. Jump step: StepStatusGood/Fail
   $$ StepStatusFail = Lib(Pega-RULES:String).inString(pxThread.pxMethodStatus, "Fail")
   $$ StepStatusGood = pxThread.pxMethodStatus = “Good”
1. Error handler flow in connect rule: Connection Problem
2. Fault Tab in Connect rule: To hold the values of ErrorCode and ErrorDesc

16. How to configure Connect-REST:

JSON Request: PolicyID
JSON Response: PolicyType, PhoneNumber
Run the wizard then it will be landed in the wizard screen.

1. Connection: Enter name and EndPointURL

$$ Why do we parameterised the resource ?
I host a service based on the geographical location. I provide one type of service to India and another type of service to USA
$$ Query string parameters is nothing but request parameter like “PolicyID”.

2. Resource method: here we can select Get, Post, Put, Delete. we have option to override the Query string and header fields
3. Data model: here we can specify the request and response parameters
4. Review: here we can specify Integration and Data layers Class name, Connector name, Ruleset

Once click on Submit we can get the below rules
Class
Property
Connect REST
Authentication profile
Data Page
Data Transform
$$ Diff between GET & POST:
In Get method, we don’t need to request body we can send the request as Query String.
In Post method, we will send the request in request body.

$$$$$$$ CONFIGURE SERVICE-REST: $$$$$$$$$

Create Data Type in application with local storage 10 records. Here, Pega acts as Service provider.
Data Table Class: Tata-FW-Data-Customer.

#1. We need to create Integration class name, Ruleset name and Ruleset Version.
Class: Tata-FW-Int-GetCustomerDetails
Ruleset: DemoFWInt

Version: 01-01-01
Note: We are creating in the framework Class Int layer so that this service can be used across all the applications.

#2. Data Mapping
Class for Request: Tata-FW-Int-GetCustomerDetails-Request ———> CustID
Class for Response: Tata-FW-Int-GetCustomerDetails-Response ———> CustomerList. CustID, CustName, CustType

#3. Create Service Package
We need to enter Service access group, Service requestor (Stateful & Stateless)
Note: Service Package is data instance which explicitly added.
Stateless: If the service method can be executed by any requestor then select Stateless
Stateful: If service method needs a requestor which already has state and clipboard data and has completed calling one of the service method belongs to the same package then select Stateful.

#4. Create Service-REST rule.
Service name: GetCustomerDetails
Service Package Name: Transport
Service Version: V1
Click create and open
Here, we need to add the class name: Tata-FW-Int-GetCustomerDetails
Endpoint URL would take the below syntax:
http(s):envdomain:portnumner/prweb/PRServlet/{ServicePackageName}/{Version}/{ServiceName}

#5. Create Service Activity
Here we need to write the business logic to fetch the details from our Data Type.
—> Page-New to create a new Page
—> Property-Set to map request params
—> Obj-Browse to fetch records from DB
—> Property-Set to map response params
—> Page-Remove to Remove the page

#6. Method requirements
Get, Post, Put, Delete

#7. Request Mapping
If we have request body we need to go for POST method.
CustID is the query string parameter that we are mapping to .Request.CustID

#8. Response Mapping
MapFrom: JSON

Map From Key: Response Attribute (Response.CustomerList)

#9. Run Service-REST rule standalone to pass request and check the response

$$$$$$$$$$$$$$
$$$$$$$$$$$$$$

17. What is Simulation:

Pega provides the ability to simulate the external data sources. Simulating an external data source is useful in situations when the external system is not ready or unavailable. You can simulate any external data source as long as you know what data the source system is expecting and returning. Simulating a data source using sample data to allows you to develop and test the application without interruption until the source system is ready.
You can also simulate your data source when the source system does not have data, or API under development.

18. Associations:

If we find, we join two classes frequently in report definition you can create an Association Rule.
It explicitly defines the Join condition between two classes.
Prefix should be same as Association rule. We need to update the Association rule name in query tab..

19. How to configure Connect-File?

The purpose of Connect-File is to create a file and place it in the mentioned app server directory.
No wizard available for Connect-File.
Create new Connect-File rule and provide the Destination path (Directory path of the app server) and File name (Name of the file to be created).

$$ How to invoke Connect-File?

1. Create a new connector activity and use Connect-File method. Param are Connector Name, Operation and Parameter
2. From Flow integration shape. Flow —> Advanced shape —> Integration utility

$$ How to debug the integration?

1. Create a file containing string and place it in App server
2. Create Connect-File
3. Create a new connector activity

20. How to export data from Pega Application to Excel file?

1. Create Case Type with configure view (Create Section rule)
2. Create an Activity with

$$ Property-Set like (Property Name as pyWorkPage.StudentName, pyWorkPage.SSCPercentage) (Property Value as Primary.StudentName, Primary.SSCPercentage)

$$ Call “pxGenerateExcelFile” and pass params (File name , Template)

3. Create Excel file like, Student name {pyWorkPage.StudentName} SSCPercentage {pyWorkPage.SSCPercentage} and save file in local drive.

4. Create Binary File rule and upload the Excel file.

5. Create section add StudentName and SSCPercentage properties. Add Download button and in the Action button add OnClick event and refresh section action add Activity.

21. How to read data from excel file and copy in to pagelist property?

1. Create Case type
2. Create an activity
   $$ Peage-New
   $$ Call “pxParseExcelFile” with parameters (FileName, Template)
   $$ Loop thru Pagelist property
   $$ Property-Set (Source: EmployeeData.pxResults(<CURRENT>).EmployeeID) and
   (Target:Primary.EmployeeInfo(<CURRENT>).EmployeeID)
3. Create two excel file. One as Template and Second as which holds data.
4. Create Binary file.
5. To show case data we may need Data Type with property defined as Pagelist.

22. How to copy one Pagelist property to another Pagelist property?

By using data transform we can do it.

1. For Each Page In —> Pagelist
2. When —> Condition is true
3. Set —> (Target: pyWorkPage.Pagelist(<APPEND>).Property)
   (Source: pyWorkPage.Pagelist(<CURRENT>).Property)

23. How to create Operators dynamically?

1. We have OOTB activity called “CreateOperator” that we need to customise this activity and need to save as to our Work class and pass UserId, Password and Username parameters.
2. Create section with Operator ID, User name and Password and we need to refer the activity in the post action of the flow action
3. Need to create Operator for the end user login so that user will login and create operators.

24. How to delete operators based on the access group?

1. We have OOTB Report Definition called “pxGetListOfOperator” that we need to save as the instance and update the filter condition pyAccessGroup equals to ABC:Admin
2. Create an activity in Work- class.
   $$ Call RD using pxRetrieveReportData and pass parameters ReportName and Class
   $$ Loop thru pyReportContent Page
   $$ Obj-Delete method to delete Operator IDs.

25. How to copy the data from Pagelist property to Page?

1. For this we need to create Pagelist property with Data class (A-B-Data) and Page property (A-B-Data)

2. Create Data transform, rule

$$ For Each Page In —> Pagelist

$$ When condition —> Primary.Pagelist(<CURRENT>).pySeleted.

$$ Set target as (Primary.Page.Property) and Source as (Pagelist(<CURRENT>).Property)

3. Define the Data transform in Post action or assignment level Set Data transform.

26. What is the purpose of Savable data page?

IMP NOTES:

1. Node level scope only applicable for readonly data pages it is not applicable for Editable and saveable data pages.

2. The order of post processing will be saveable data page execution followed by data transform and activity.

3. You cannot directly save on data page pxResults(). You should copy into some other page and save.

4. Never use commit method directly in the data base save activity because this may break the transaction which gives undesired results.

$$ With Savable data pages we can combine both business logic (Fetch data) and Transactional data (save data) of any given scenario.

$$ You can use database save option to save single instance and activity to save the list of data instances.

$$ Savable data pages can be explicitly triggered from flow, flow action post processing and activity method Save-datapage.

$$ Database save option passes and uses the WriteNow parameter automatically. For activity based data save use need to use it conditionally write or defer save.

$$ This can be applicable only for Thread and Requestor level data pages and also applicable for Page and List type structures.

1. To configure Savable data page, create Savable data page as list.

2. Create source DT to append values into D_SavableDP.pxResults from Paglist property.

$$ Remove the D_SavableDP.pxResults

$$ Append to —> Source as (pyWorkPage.Pagelist) and Target as (D_SavableDP.pxResults)

3. Create activity as data save options to save values into local data storage from D_SavableDP.pxResults

$$ Page-New (define steppage)

$$ Loop thru D_SavableDP.pxResults and do Page-Copy from D_SavableDP.pxResults(<CURRENT>) to
StepPage of Activity.

$$ Obj-Save and Commit.

4. Update data on UI and click submit. The updated data will be saved into DB.

27. How to send an email to customer post login..

1. First we need to create an Email Account to send an email. Configure —> Integration —> Email —>Email wizard
2. Enter the account information like, Account Provider, Sender details like Identity and Connection
3. Enter Receiver details and click submit. Do the test connection for successful configuration.
4. In flow, we need to select Send Email (shape) and provide the configurations and save the flow.
5. Run the flow and check it in mail box.

$$$ To send EXCEL file as an attachment

1. We need to configure the utility shape in the flow.
2. Call the Report Definition to fetch the records and put it on page
3. Call pxGenerateExcelFile activity to generate excel file (params: FileName, Template)
4. We need to configure Attachment page on Data-WorkAttach-File (Table: pc_data_workattach)

28. Asynchronous data page:

$$ In activity we have 'Load-Datapage' method to load the data asynchronoulsy. We can specify the PoolID for reference in load data page method.

$$Data pages can be loaded asynchronously using Load-DataPage and Connect-Wait.

$$ Imagine the call centre application, when the agent engages with the customer thru phone he needs to get the customer details before him. We may need to integrate with other applications to get data with can affect performance during loading.

$$ When the load data page gets executed, the data page loaded in background thread and the activity continue processing without waiting for data page loading.

1. Load-DataPage: This method used to trigger (Queue) the data page execution in Asynchronous mode.

Parameters: PoolID, Data Page

2. Connect-Wait: This method is used to ensure the execution of the queued data page is complete and make it ready for access.

Parameters: Wait seconds, Pool ID

29. Linked Properties:

$$ Linked properties are single value text properties defined in one concrete class that can establish read-only link to a record in different concrete class.

$$ It can be linked there is a need to link records from two different concrete classes.

How to use Linked properties:

1. Create a single value text property “Customer” in ABC-Data-Order.
2. In the Data Access section in the property select “Automatic reference to class instance (linked).
3. Give the linked class as ABC-Data-Customer class. In the linked mapping use CustomerID which is primary key of Customer lass.
4. To access the “CustomerName” we can use the syntax as “Customer.CustomerName”.

30. How to perform Duplicate Search in Pega:

$$ Pega Case Management provides us with an option of “Search Duplicate Cases” which directly configured as step in the desired stage to identify potential duplicates between cases that are already created in the system.

$$ To evaluate duplicate cases we need to configure both Basic condition and weighted condition.

31. Dynamic Class Reference:

$$ DCR is a design pattern to avoid hard coding of class names, flow names and work basket names.

How to configure DCR:

1. First we need to create “AppExtension” single page and pass data class as Page definition.
2. Select “Refer to DataPage” radio button and refer D_AppExtension data page.
3. Create Data Transform to hold reference values like,
   “ClaimWorkClass —> ABC-CS-Work-Claims” and
   “InsuranceWorkClass —> ABC-CS-Work-Insurance”
4. Refer Data transform in Data Page as source.
5. Use it in Obj-Open method like, “D_AppExtension.ClaimWorkClass”

32. Authentication & Authorisation:

$$ Authentication confirms when you login to PC using user id and password. Pega platform includes user logins, Pega platform request to external services and external services requests to Pega platform.

There are two types of Authentications:

1. Basic authentication —> provide user id and password
2. SAML SSO authentication: Identity provider (oAuth.com) and service provider

$$ Authorisation is the process of verifying that you’re accessing some thing with role based or attribute based controls. It ensures that after user login, they have only access to platform features and data that they need for their work.

Authorisation offers three types: Role based, Attribute based and Client based.

33. Role based and Access based and Client based access controls:

RBAC: To restrict users from having access to certain UI elements to perform certain actions in the UI or having any access to class based on defined roles and privileges that are derived from access group.

ABAC: To restrict the ability of user to view, modify and delete an instances of classes or properties within the classes.

CBAC: To track the process requests to view, change or remove the data.

34. Difference between ARO and Access Deny:

ARO: It specify permission that are granted to role and access class. This rule applies to all instances of its access class

Access Deny: To restrict users who have a specified access role from accessing instances of specific class under certain conditions. Daniel of access to class can depend on the production level value of 1 to 5 of your system.

$$ Scenario: We have supervisor access group and it has three roles: User, Approver and Manager and we need to restrict particular class instance.

Sol 1: Use ARO and make sure ARO's in all three roles should be restrict to access level 0.

Sol 2: Use access deny rule and update any one access role with access deny restrictions.

Note: Access deny would take precedence over ARO in case both are true.

35. How to create and manage Temporary work objects:

$$ Scenario: Consider reimbursement application there is a duplicate check to handle to if the reimbursement claim is duplicate. In this scenario we can initially create temporary work object, perform duplicate search and then make it permanent only if it is non duplicate case.

1. To create a temporary work object goto CaseType —> Settings —> General category —> "Create temporary case that is not saved until a persist case step is reached" enable

2. Once this option is enabled and saved we can notice that pyStartCase flow for Temporary Object check box enabled.

3. We can use "Persist Case" shape or option to make Temporary work object to permanent in the database.

36. Difference between Pega logs and Alert logs:

$$ PEGA LOGS written by system when ever exception occurs in our environment. It contains warnings, errors and information about internal operations.

EX: Null pointer exception or Array out of bound exception.

$$ ALERT LOGS contains the performance related alerts. Use PAL tool to parse the file into MS excel spreadsheet.

$$ How do we resolve the slowness issues in Production application?

1. we will check the memory utilisation first. we have third party apps called App Dynamics.

2. We need to check what alerts we are getting in Alert logs. If we found

any alert triggering continuously we need to give the fix for that.

3. If it requires server restart we need to restart the server.

37. Requestor types:

$$ BROWSE:

Normally developers or end users access Pega platform using browsers i.e, IE or Chrome. As long as they hit the URL requestor type of Browser is created.

$$ BATCH:

Internal back ground process comes under Batch requestor type.

We have two types of Agents.

Standard: Standard Agent completely ignores this field. It uses the access group of the Operator which queues an entry to the Standard Agent.

Advanced: Pega uses the access group specified in the Security tab to determine the application and locate the agent activity rule.

If access group in the security tab empty then Pega uses the access group specified in the browser requestor type to determine the application and locate the agent activity rule.

$$ APPLICATION:

Typically Listeners and External Services comes under Application requestor type. Both listeners and external services included under Service Package.

$$ PORTAL:

Pega users access group specified in the Portlet service package to determine the application.

38. How to Expose property in Pega:

$$ Exposing a property is nothing but creating a dedicated column for the property in the database table.

$$ By default, all the elements will be persisted in a BLOB column (pxPVStream) in a compressed form. Such columns cannot be used in selection of reporting.

There are 4 ways to expose a property

1. Using Property rule form

2. Using modify schema wizard: Designer studio —> System —> Database —> Modify Schema

3. Directly create column in data base: select paid, name, age FROM tablename where paid == 'A1';

4. Expose property using declare index rule

39. Tickets vs Change stage:

1. Tickets can be used to jump to or go to another path of the current sequence of execution. Tickets do not have any processing logic.

2. Change stage can be used to move to different stage from the current stage, while, tickets are used to move back and forth between flows that are currently present in the pxFlow Page group.

3. It is always a good practice to use SetTicket rather than using Obj-Set-Tickets since SetTicket takes care of both setting and removing ticket.

40. Declare Index:

$$ It is used to expose aggregate properties like Page/list/group for reporting purpose. It fired automatically without any user invocation.

$$ when we expose property inside page, Pega will expose it as single value column and adds it to the external mapping tab of the class rule. To avoid this we might need to create index- class explicitly.

1. Create new Index table
2. Create new index class including pxInsIndexedKey, pxIndexCount, pxIndexPurpose
3. Create new properties (index class) that we need to expose in index table.
4. Create new database table instance and map the class with index table created in DB
5. Create declare index rule and configure all the details.

NOTE: Use OOTB activity "RecreateIndexesForClass" to fire declare index for the inflight cases.

41. How many pages are available in clipboard page:

1. User Page
2. Data Page
3. System Page
4. Linked property pages

42. Data Transform methods:

1. Set:

This is simple and most commonly used to set any data.

EX: For Value list and Value group properties we can directly set values with out using Subscripts.

PhoneNumber(1) equals to SourceList(1).PhoneNumber(1) to copy the entire Valueless content.

2. Remove:

This can be used to remove the top level page or property value in the clipboard. Similar to Page-Remove method.

EX: Remove —> Sourcelist

3. Update Page:

This action helps us to update any page in the clipboard.

EX: we are going to copy the entire Page list property from

Update Page —> ActiveTestPage (Target) with values from ActivePage (Source)

Set —> Targetlist equals to Sourcelist

Which means, ActivePage.Sourcelist to ActiveTestPage.Targetlist

If we want to update Primary page we no need to use above action.

4. Append to:

It helps to append a Page to the Pagelist property. here both Source and Target should be in same class.

EX: we have Pagelist property Sourcelist contains 2 pages. I need to insert new page to the page list

Append to —> Sourcelist(Target) —> a new page [new page will be added like Sourcelist(3)]

Append to —> Sourcelist(Target) —> an existing page ActiveTestPage [ActiveTestPage values Append to Sourcelist page]

Append to —> Sourcelist(Target) —> current source page [Primary page context]

Append to —> Targetlist(Target) —> each page in —> Sourcelist [Appending the Sourcelist with Targetlist]

5. Append and map to:

We use this mainly to copy data from one page to another page without Embedded properties and with different applies to class.

EX: Append and map to —> Targetlist (Target) —> each page in Sourcelist(Source)

Set —> pyLabel equals to “ABC” [Targetlist set to pyLebel]

$$ Diff between Append to and Append and map to:

1. Append to is the target and source can be of same applies to class whereas Append and map to target and source applies to different class.

2. Append and map to create a new page without embedded properties while Append to action create new page with all values form Source page.

IMP NOTES:

1. you can never persist data using DT

2. Append to and Append and map to action don’t work for Page group properties. Only for Pagelist properties.

43. Obj- Open and Obj-refresh and lock:

$$ Obj-Open: It is used to open single record based on class key. For read only purpose you no need to lock it but for updating purpose you need to lock it. Lock is optional when we use Obj-open.

Suppose If we are checked the LOCK check box that means lock is acquired on current object and we have maintain the exception handling.

$$ Obj-Refresh and lock: If lock is already acquire on the respective record or class where the step page has some key value this method has no impact.

Suppose lock is not there, it will open the record and freshly reload this page with latest data acquire lock and proceed further.

So, exception handling mechanism provided by the Obj-Open method in case of locking, then we have method called Obj-refresh and lock to refresh the data.

44. Difference Obj-browse and Report definition:

1. Obj-browse not filter on properties which are not optimised whereas Report definition have filter on properties.
2. Obj-browse will read the BLOB properties whereas Report definition will not read the BLOB data
3. Obj-browse doesn't return the data from multiple tables whereas Report definition will return data from multiple tables using JOINS.

45. Differ between Parameter page and Prompt page:

$$ Parameter page is a single page containing all parameters that are passed into a rule. It is not the same as a clipboard page and does not exist on the clipboard, but offers much of the same functionality. Parameter pages are mostly handled behind the scenes for you based on your preferences on the relevant "Parameters" tabs, and whether you check the "Pass current parameter page" checkbox when calling an Activity from another Activity.

If you set a parameter to type "Page Name" it can be used as the name of a page, thus allowing you to pass an entire clipboard page as a parameter.

$$ Prompt Page is another way to pass a page into an Activity. If a page is set to 'Prompt' mode, then another textbox appears below the "Call" step where you can specify the name of the page to be passed to the called activity. Prompt Pages were available before "Page Name" Parameters.

Since "Page Name" parameters offer the same functionality, I believe prompt/indirect pages are considered deprecated. If I find an authoritative source that says so, I'll update this post.

46. Performance tools in Pega:

$$ PAL: The Performance Analyzer (PAL) **provides a view to all the performance statistics that Pega Platform captures**. You can use PAL to understand the system resources consumed by processing a single requestor session.

Performance is a function of how each of the component performs with in the application.

PRPC includes the following:

1. Servers might be responding slow
2. Unoptimised DB issues.
3. Memory management
4. UI design like loading data in a screen without pagination
5. Clipboard size

47. Obj vs RDB methods:

OBJ methods can update the BLOB properties

RDB methods can update only exposed properties

OBJ methods have deferred save option which means delay committing.
RDB methods commits as soon as the step got executed
OBJ methods system is responsible for form a query
RDB methods need to form an external query using connect-sql rule
OBJ methods always used to query single instance
RBD methods are used to query multiple instances

48. Process flow vs Screen flow:

Process flow can be routed to many people to work on
Screen flow single people to work on
Process flow routing can be configured in assignment shape
Screen flow routing can be configure in start shape
Process flow we can specify harness in different forms
Screen flow we can specify harness in start shape
Process flow can specify flow actions in the connector shape
Screen flow can specify flow actions in the assignment share
Process flow don't have option to go back and forth
Screen flow will have option to go back and forth

49. How can we improve the Pega application performance:

1. Use data pages when possible
2. Pagination results
3. Optimising properties
4. Using declare index to expose embedded properties
5. Use reports to fetch data from DB
6. Avoiding Outer join: include all rows option

50. Pega Production env slowness issue.

$$ System admin has attempted to shut down or restart the env, and perform some basic maintenance on DB. But unable to improve the system performance.

$$ No specific error message found in the pega logs.

$$ but sometimes memory consumption problem may occurs. To resolve this we need to increase the minimum heap space to 6 GB size in production env.